

Binary File Handling in Java

29 June 2026 20:30

A binary file stores data in binary (zeros and ones) Rather than as human readable text. Unlike text files, binary files preserve the exact internal representation of data. Making them faster, more compact, and suitable for storing objects, images, audio, video and primitive data types.

For example:

Text File(student.txt)

```
101 Amit 92.5
102 Priya 88.0
```

The contents are readable

Binary File(student.dat)

```
010000001 0110101101 0111101110...
```

The content is not human readable. Sometime it displays symbols also, when we render it on different text editors.

Advantages of Binary Files

- Faster read or write operation.
- Occupy less storage.
- Preserve the exact data type.
- No conversion between text and numeric values.
- Suitable for storing Java objects.

Java Classes Used for Binary File Handling

Java provides several classes for binary file operations.

Class	Purpose
FileInputStream	Reads binary data byte by byte.
FileOutputStream	Writes binary data byte by byte.
BufferedInputStream	Faster buffered reading.
BufferedOutputStream	Faster buffered writing.
DataInputStream	Reads primitive data types.
DataOutputStream	Writes primitive data types.
ObjectInputStream	Reads serialized objects.
ObjectOutputStream	Writes serialized objects.
RandomAccessFile	Reads or writes at any location.

FileOutputStream: We used to write raw bytes into a binary file.

Class FileOutputStream

```
java.lang.Object
  java.io.OutputStream
    java.io.FileOutputStream
```

All Implemented Interfaces:

Closeable, Flushable, AutoCloseable

```
public class FileOutputStream
extends OutputStream
```

A file output stream is an output stream for writing data to a `File` or to a `FileDescriptor`. Whether or not a file is available or may be created depends upon the underlying platform. Some platforms, in particular, allow a file to be opened for writing by only one `FileOutputStream` (or other file-writing object) at a time. In such situations the constructors in this class will fail if the file involved is already open.

`FileOutputStream` is meant for writing streams of raw bytes such as image data. For writing streams of characters, consider using `FileWriter`.

API Note:

The `close()` method should be called to release resources used by this stream, either directly, or with the try-with-resources statement.

Implementation Requirements:

Subclasses are responsible for the cleanup of resources acquired by the subclass. Subclasses requiring that resource cleanup take place after a stream becomes unreachable should use `Cleaner` or some other mechanism.

Since:

1.0

Constructor Summary

Constructors

Constructor	Description
<code>FileOutputStream(File file)</code>	Creates a file output stream to write to the file represented by the specified <code>File</code> object.
<code>FileOutputStream(FileDescriptor fdObj)</code>	Creates a file output stream to write to the specified file descriptor, which represents an existing connection to an actual file in the file system.
<code>FileOutputStream(File file, boolean append)</code>	Creates a file output stream to write to the file represented by the specified <code>File</code> object.
<code>FileOutputStream(String name)</code>	Creates a file output stream to write to the file with the specified name.
<code>FileOutputStream(String name, boolean append)</code>	Creates a file output stream to write to the file with the specified name.

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
void	<code>close()</code>	Closes this file output stream and releases any system resources associated with this stream.
<code>FileChannel</code>	<code>getChannel()</code>	Returns the unique <code>FileChannel</code> object associated with this file output stream.
final <code>FileDescriptor</code>	<code>getFD()</code>	Returns the file descriptor associated with this stream.
void	<code>write(byte[] b)</code>	Writes <code>b.length</code> bytes from the specified byte array to this file output stream.
void	<code>write(byte[] b, int off, int len)</code>	Writes <code>len</code> bytes from the specified byte array starting at offset <code>off</code> to this file output stream.
void	<code>write(int b)</code>	Writes the specified byte to this file output stream.

```
package FileHandling.BinaryFileHandling;
import java.util.*;
import java.io.*;
```

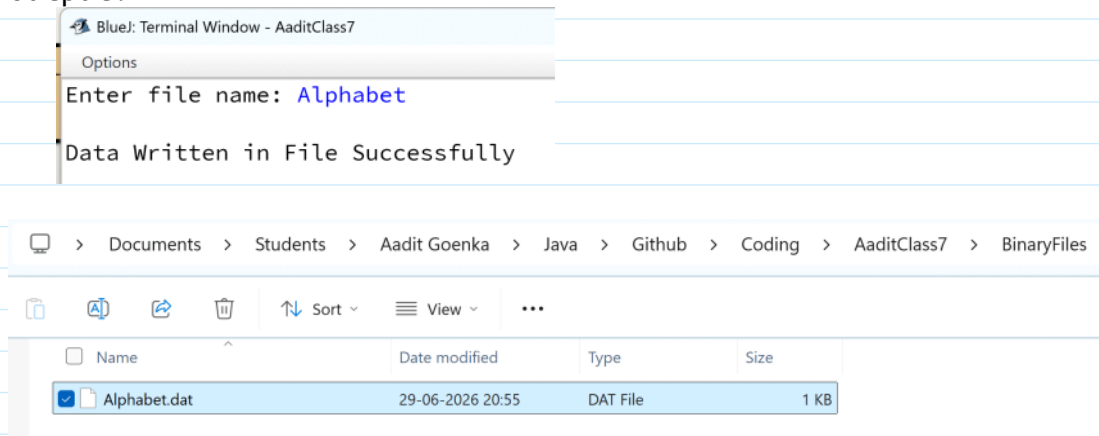
```

/**
 * Write a description of class WriteBinary here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class WriteBinary
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter file name: ");
        String path = "./BinaryFiles/";
        String fileName = sc.nextLine();
        try(FileOutputStream fos = new
        FileOutputStream(path+fileName+".dat", true)){
            fos.write(65);
            fos.write(66);
            fos.write(67);
            fos.write(68);

            fos.close();
            System.out.print("\nData Written in File Successfully");
        }
        catch(FileNotFoundException fnfe){
            System.err.print("\nFile is not present on the specified
directory.");
        }
        catch(IOException ioe){
            System.err.print("\nInput/Output network jam.");
        }
    }
}

```

Output:



FileInputStream: Reads bytes from the binary file.

Class FileInputStream

java.lang.Object
java.io.InputStream
java.io.FileInputStream

All Implemented Interfaces:

Closeable, AutoCloseable

```
public class FileInputStream  
extends InputStream
```

A FileInputStream obtains input bytes from a file in a file system. What files are available depends on the host environment.

FileInputStream is meant for reading streams of raw bytes such as image data. For reading streams of characters, consider using FileReader.

API Note:

The `close()` method should be called to release resources used by this stream, either directly, or with the try-with-resources statement.

Implementation Requirements:

Subclasses are responsible for the cleanup of resources acquired by the subclass. Subclasses requiring that resource cleanup take place after a stream becomes unreachable should use `Cleaner` or some other mechanism.

Since:

1.0

Constructor Summary

Constructors

Constructor	Description
<code>FileInputStream(File file)</code>	Creates a FileInputStream to read from an existing file represented by the File object file.
<code>FileInputStream(FileDescriptor fdObj)</code>	Creates a FileInputStream by using the file descriptor fdObj, which represents an existing connection to an actual file in the file system.
<code>FileInputStream(String name)</code>	Creates a FileInputStream to read from an existing file named by the path name name.

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
int	<code>available()</code>	Returns an estimate of the number of remaining bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream.
void	<code>close()</code>	Closes this file input stream and releases any system resources associated with the stream.
FileChannel	<code>getChannel()</code>	Returns the unique FileChannel object associated with this file input stream.
final FileDescriptor	<code>getFD()</code>	Returns the FileDescriptor object that represents the connection to the actual file in the file system being used by this FileInputStream.
int	<code>read()</code>	Reads a byte of data from this input stream.
int	<code>read(byte[] b)</code>	Reads up to b.length bytes of data from this input stream into an array of bytes.
int	<code>read(byte[] b, int off, int len)</code>	Reads up to len bytes of data from this input stream into an array of bytes.
byte[]	<code>readNBytes(int len)</code>	Reads up to a specified number of bytes from the input stream.
long	<code>skip(long n)</code>	Skips over and discards n bytes of data from the input stream.

read

```
public int read()  
    throws IOException
```

Reads a byte of data from this input stream. This method blocks if no input is yet available.

Specified by:

read in class `InputStream`

Returns:

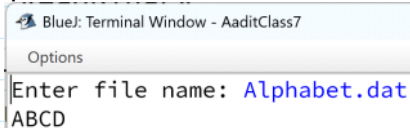
the next byte of data, or -1 if the end of the file is reached.

Throws:

`IOException` - if an I/O error occurs.

```
package FileHandling.BinaryFileHandling;  
import java.util.*;  
import java.io.*;  
  
/**  
 * Write a description of class ReadBinary here.  
 *  
 * @author (your name)  
 * @version (a version number or a date)  
 */  
public class ReadBinary  
{  
    public static void main(String arga[]){  
        Scanner sc = new Scanner(System.in);  
        System.out.print("\nEnter file name: ");  
        String path = "./BinaryFiles/";  
        String fileName = sc.nextLine();  
        int ch;  
        try(FileInputStream fis = new FileInputStream(path+fileName)){  
            while((ch = fis.read()) != -1){  
                System.out.print((char)ch);  
            }  
        }  
        catch(IOException ioe){  
            System.out.print("\nNetwork error!");  
        }  
    }  
}
```

Output:



```
BlueJ: Terminal Window - AaditClass7  
Options  
Enter file name: Alphabet.dat  
ABCD
```

Writing Primitive Data to a Binary File

Suppose we want to store:

- Roll Number (int)

- Name(string)
- Marks(float)

Class DataOutputStream

java.lang.Object
java.io.OutputStream
java.io.FilterOutputStream
java.io.DataOutputStream

All Implemented Interfaces:
Closeable, DataOutput, Flushable, AutoCloseable

public class DataOutputStream
extends FilterOutputStream
implements DataOutput

A data output stream lets an application write primitive Java data types to an output stream in a portable way. An application can then use a data input stream to read the data back in. A data output stream wraps another output stream and delegates writing bytes to the write methods of that output stream. Writing data consisting of more than a single byte may cause several writes to the underlying output stream.

A DataOutputStream is not safe for use by multiple concurrent threads. If a DataOutputStream is to be used by more than one thread then access to the data output stream should be controlled by appropriate synchronization.

Since:
1.0

Constructors

Constructor	Description
DataOutputStream(OutputStream out)	Creates a new data output stream to write data to the specified underlying output stream.

Method Summary

All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
void	flush()	Flushes this data output stream.
final int	size()	Returns the current value of the counter written, the number of bytes written to this data output stream so far.
void	write(byte[] b, int off, int len)	Writes len bytes from the specified byte array starting at offset off to the underlying output stream.
void	write(int b)	Writes the specified byte (the low eight bits of the argument b) to the underlying output stream.
final void	writeBoolean(boolean v)	Writes a boolean to the underlying output stream as a 1-byte value.
final void	writeByte(int v)	Writes out a byte to the underlying output stream as a 1-byte value.
final void	writeBytes(String s)	Writes out the string to the underlying output stream as a sequence of bytes.

final void	<code>writeChar(int v)</code>	Writes a char to the underlying output stream as a 2-byte value, high byte first.
final void	<code>writeChars(String s)</code>	Writes a string to the underlying output stream as a sequence of characters.
final void	<code>writeDouble(double v)</code>	Converts the double argument to a long using the <code>doubleToLongBits</code> method in class <code>Double</code> , and then writes that long value to the underlying output stream as an 8-byte quantity, high byte first.
final void	<code>writeFloat(float v)</code>	Converts the float argument to an int using the <code>floatToIntBits</code> method in class <code>Float</code> , and then writes that int value to the underlying output stream as a 4-byte quantity, high byte first.
final void	<code>writeInt(int v)</code>	Writes an int to the underlying output stream as four bytes, high byte first.
final void	<code>writeLong(long v)</code>	Writes a long to the underlying output stream as eight bytes, high byte first.
final void	<code>writeShort(int v)</code>	Writes a short to the underlying output stream as two bytes, high byte first.
final void	<code>writeUTF(String str)</code>	Writes a string to the underlying output stream using <code>modified UTF-8</code> encoding in a machine-independent manner.

Reading Primitive Data

Class `DataInputStream`

```
java.lang.Object
  java.io.InputStream
    java.io.FilterInputStream
      java.io.DataInputStream
```

All Implemented Interfaces:

`Closeable`, `DataInput`, `AutoCloseable`

```
public class DataInputStream
  extends FilterInputStream
  implements DataInput
```

A data input stream lets an application read primitive Java data types from an underlying input stream in a machine-independent way. An application uses a data output stream to write data that can later be read by a data input stream.

A `DataInputStream` is not safe for use by multiple concurrent threads. If a `DataInputStream` is to be used by more than one thread then access to the data input stream should be controlled by appropriate synchronization.

Since:

1.0

Constructor Summary

Constructors

Constructor	Description
<code>DataInputStream(InputStream in)</code>	Creates a DataInputStream that uses the specified underlying InputStream.

Method Summary

All Methods	Static Methods	Instance Methods	Concrete Methods	Deprecated Methods
Modifier and Type	Method	Description		
final int	<code>read(byte[] b)</code>	Reads some number of bytes from the contained input stream and stores them into the buffer array b.		
final int	<code>read(byte[] b, int off, int len)</code>	Reads up to len bytes of data from the contained input stream into an array of bytes.		
final boolean	<code>readBoolean()</code>	See the general contract of the readBoolean method of DataInput.		
final byte	<code>readByte()</code>	See the general contract of the readByte method of DataInput.		
final char	<code>readChar()</code>	See the general contract of the readChar method of DataInput.		
final double	<code>readDouble()</code>	See the general contract of the readDouble method of DataInput.		
final float	<code>readFloat()</code>	See the general contract of the readFloat method of DataInput.		
final void	<code>readFully(byte[] b)</code>	See the general contract of the readFully method of DataInput.		
final void	<code>readFully(byte[] b, int off, int len)</code>	See the general contract of the readFully method of DataInput.		
final int	<code>readInt()</code>	See the general contract of the readInt method of DataInput.		
final String	<code>readLine()</code>	Deprecated. This method does not properly convert bytes to characters.		
final long	<code>readLong()</code>	See the general contract of the readLong method of DataInput.		
final short	<code>readShort()</code>	See the general contract of the readShort method of DataInput.		
final int	<code>readUnsignedByte()</code>	See the general contract of the readUnsignedByte method of DataInput.		
final int	<code>readUnsignedShort()</code>	See the general contract of the readUnsignedShort method of DataInput.		
final String	<code>readUTF()</code>	See the general contract of the readUTF method of DataInput.		
static final String	<code>readUTF(DataInput in)</code>	Reads from the stream in a representation of a Unicode character string encoded in modified UTF-8 format; this string of characters is then returned as a String.		
final int	<code>skipBytes(int n)</code>	See the general contract of the skipBytes method of DataInput.		

```
package FileHandling.BinaryFileHandling;
import java.io.*;
import java.util.*;

/**
 * Write a description of class ReadPrimitiveData here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ReadPrimitiveData
{
    public static void main(String args[]){
```

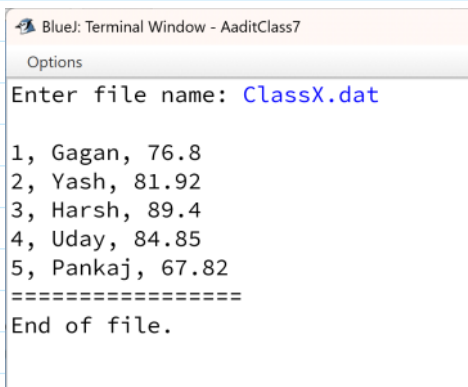
```

Scanner sc = new Scanner(System.in);
System.out.print("\nEnter file name: ");
String path = "./BinaryFiles/";
String fileName = sc.nextLine();
String pathAndFile = path + fileName;
try(FileInputStream fis = new FileInputStream(pathAndFile);
    DataInputStream dis = new DataInputStream(fis)){
    boolean more = false;
    int rn;
    String name;
    float marks;

    try{
        while(true){
            rn = dis.readInt();
            name = dis.readUTF();
            marks = dis.readFloat();
            System.out.print("\n"+rn+", "+name+", "+marks);
        }
    }
    catch EOFException eofe){
        System.out.print("\n=====");
        System.out.print("\nEnd of file.");
    }
}
catch(IOException e){
    e.printStackTrace();
}
}
}

```

Output:



```

BlueJ: Terminal Window - AaditClass7
Options
Enter file name: ClassX.dat

1, Gagan, 76.8
2, Yash, 81.92
3, Harsh, 89.4
4, Uday, 84.85
5, Pankaj, 67.82
=====
End of file.

```

Important: Read data in the same order in which it was written. Otherwise, the program may throw exceptions or produce incorrect values.

Homework do Any 2 Question based on text file using binary files.